



CAPÍTULO 6

Vetores e matrizes

Como o C guarda várias posições sob um único nome — e por que, por baixo dos panos, tudo é sempre um bloco de memória contíguo.



O que vamos ver

01 Vetores

memória contígua, índices — e os limites que o C não confere

02 Vetores e ponteiros

por que $v[i]$ e $*(v + i)$ são exatamente a mesma coisa

03 Matrizes

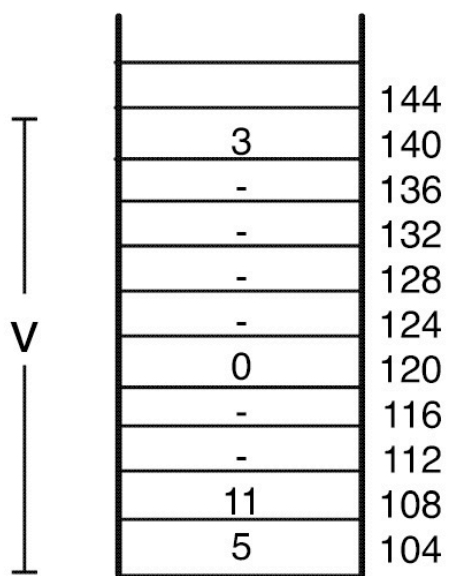
uma grade lógica sobre um único bloco de memória linear

04 Laços aninhados

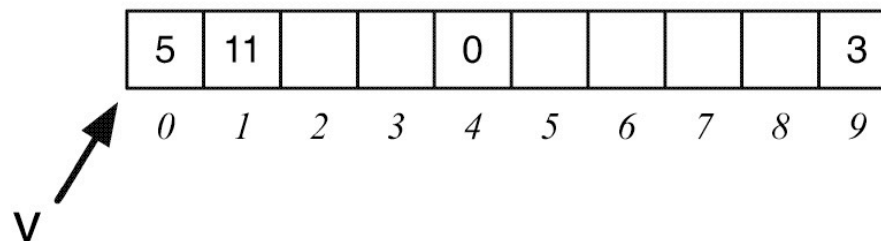
como percorrer — e nem sempre percorrer tudo — uma matriz

Um bloco contíguo, acessado por índice

`int v[10];` reserva de uma vez 10 posições — `v`
aponta sempre para a posição 0.



(a)



(b)

C não confere se o índice cabe no vetor

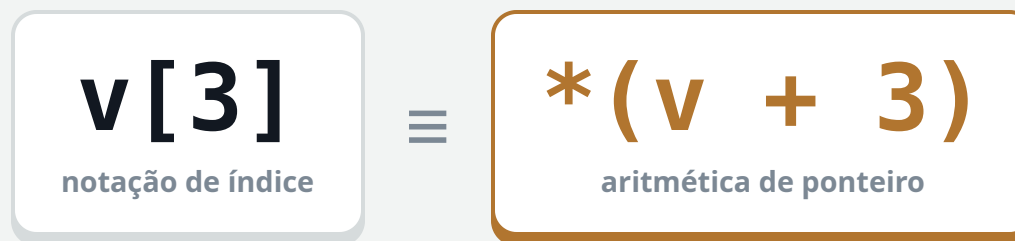
Escrever em `v[30]` com um vetor de 10 posições compila e roda — só que em memória que não é sua.



`v[0]` a `v[9]` são válidos — `v[30]` já não pertence a `v`, mas o C acessa mesmo assim.

Colchetes são aritmética de ponteiro disfarçada

O nome do vetor já é um ponteiro constante para sua primeira posição.



`sizeof(vetor) ≠ sizeof(ponteiro)`

`int v[10]; int *u = v;` — só `v` conhece o tamanho do bloco inteiro.

40 bytes

`sizeof(v)` — o vetor inteiro

VS

8 bytes

`sizeof(u)` — só um endereço

Duas formas de escrever o mesmo laço

A equivalência entre índice e aritmética de ponteiro abre dois estilos de percurso.

`*(ptr + i)`

`ptr` fica parado em `numbers[0]` do início ao fim — é o deslocamento + `i` que muda a cada volta.

`*ptr; ptr++;`

Sem índice: `*ptr` lê o valor atual, depois `ptr++` avança o próprio ponteiro.

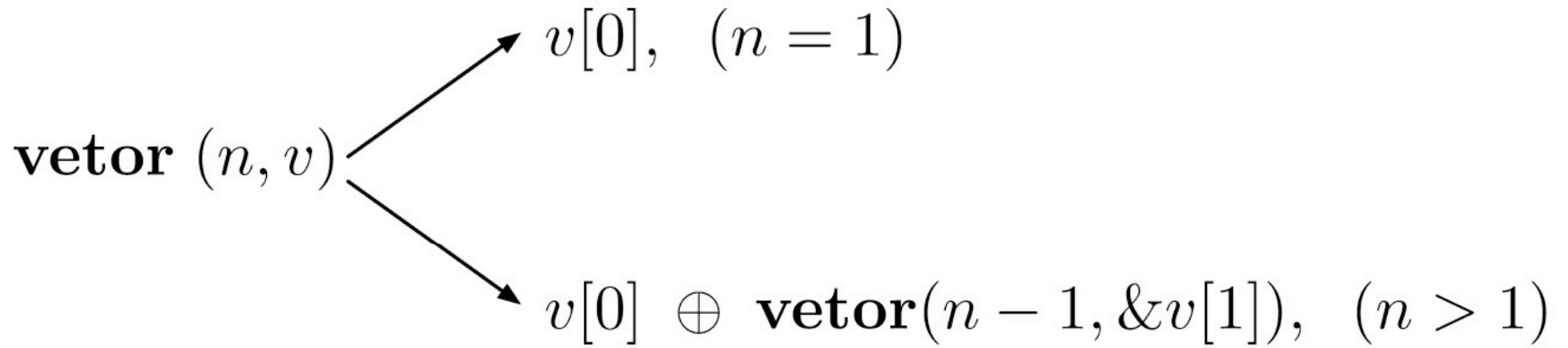
Passar um vetor é passar um endereço

`float calculate_mean(float ptr[], int size)` — dentro da função, `ptr` é só um ponteiro comum.



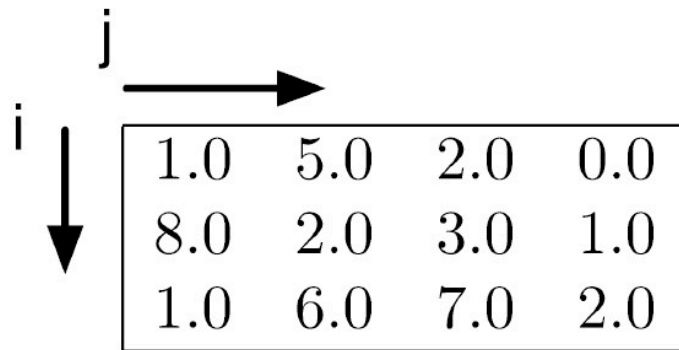
O mesmo padrão, um elemento menor a cada chamada

Caso base: um único elemento. Caso recursivo: o primeiro combinado com o resultado do resto.



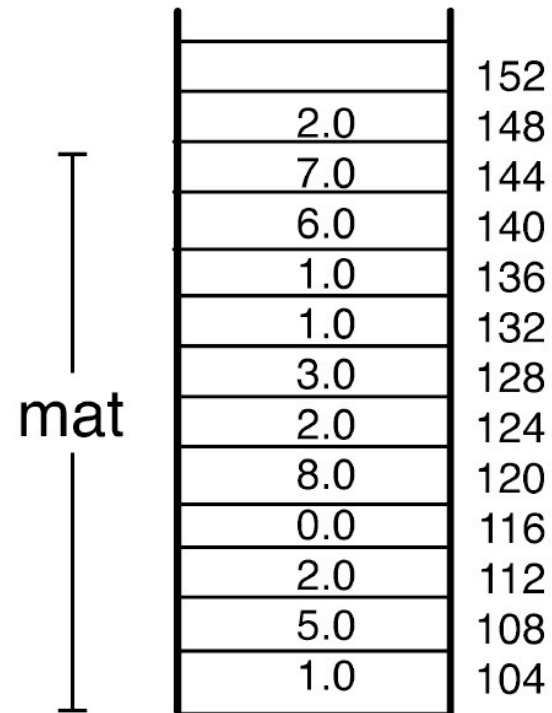
Uma grade lógica sobre um bloco linear

`int mat[3][4];` guarda 12 inteiros contíguos,
preenchidos linha por linha.



A 3x4 grid of values. To the left of the grid is a vertical arrow pointing downwards labeled 'i'. Above the grid is a horizontal arrow pointing to the right labeled 'j'.

1.0	5.0	2.0	0.0
8.0	2.0	3.0	1.0
1.0	6.0	7.0	2.0



A vertical column of 12 values. To the left of the column is a vertical line with horizontal caps at both ends, labeled 'mat'.

	152
2.0	148
7.0	144
6.0	140
1.0	136
1.0	132
3.0	128
2.0	124
8.0	120
0.0	116
2.0	112
5.0	108
1.0	104

Um laço para a linha, outro para a coluna

O laço externo só avança para a próxima linha depois que o interno termina todas as colunas.

	j=0	j=1	j=2	j=3
i=0	1	5	2	0
i=1	8	2	3	1
i=2	1	6	7	2

Ordem de `mat[i][j]` : 0,0 · 0,1 · 0,2 · 0,3 · 1,0 ...

Nem sempre é preciso visitar tudo

A diagonal principal ($i == j$) precisa de um único laço, sem aninhamento.

	j=0	j=1	j=2	j=3
i=0	1	5	2	0
i=1	8	2	3	1
i=2	1	6	7	2

Soma apenas onde $i == j$: $1 + 2 + 7 = 10$

Linhas livres, colunas obrigatórias

```
int primary_diagonal_sum(int matrix[]  
[4], int size) — só o número de colunas  
precisa ser fixo.
```

matrix[]

linhas — pode ficar em branco

[4]

colunas — obrigatório

AGORA É COM VOCÊ

Explore os exemplos, no seu ritmo

Código-fonte, explicação linha a linha e simulação da execução passo a passo — sem precisar compilar nada.

 **Notas de Aula e Exemplos** thiagopx.github.io/ed/04_arrays

Depois, teste o que aprendeu no quiz — 10 questões sobre vetores, ponteiros e matrizes.